CSA4701-DEEP LEARNING

NAME : YUGITHA B

REG NO : 192424102

ASSIGNMENT 2 – DEEP RECURRENT NETWORK PERFORMANCE & TUNING STUDY

Assessment topic: Train a simple RNN, a gated RNN, and a deep recurrent network on a sequence task. Evaluate using suitable metrics such as F1-score, RMSE, MAE, or perplexity. Conduct at least three controlled tuning experiments and explain when a recursive network is preferable.

TASK 1 – Define the Aim and Sequence Problem

Aim: To compare a simple RNN, a gated recurrent network (LSTM or GRU), and a deep recurrent network on the same sequence task, and to investigate how selected hyperparameters affect performance.

Example task: sentiment classification of short text reviews. Each review is a sequence of words, and the model predicts positive or negative sentiment. A time-series forecasting task is also acceptable, but one task must be used consistently for all model comparisons.

TASK 2 – Prepare the Sequence Dataset

1. Clean text consistently and tokenize it into words or tokens.
2. Convert tokens into integer IDs and pad or truncate sequences to a fixed length.
3. Create training, validation and test sets. For time-series data, preserve chronological order and avoid using future values to predict the past.
4. Use identical data splits and comparable preprocessing for all models.
5. Record dataset size, sequence length, number of classes or target values, and preprocessing steps.

TASK 3 – Design the Three Models

Simple RNN: It processes one time step at a time and carries a hidden state. A basic update is $h_t = \tanh(W_{xh} x_t + W_{hh} h_{(t-1)} + b)$. It is a useful baseline for short sequences.

Gated RNN: Use either LSTM or GRU and state which one was selected. Gates help the network decide what information to keep or forget, which can help it learn longer-term dependencies.

Deep recurrent network: Stack two or more recurrent layers. Earlier layers can learn simpler sequence patterns and later layers can learn more complex patterns. Dropout between layers may reduce overfitting.

TASK 4 – Explain the Training Method and Metrics

Train all models using the same training data and evaluate them on the same held-out test data. For classification, report accuracy, precision, recall and F1-score. For numerical forecasting, report MAE and RMSE. For language modelling, perplexity may be appropriate.

MAE = (1/n) Σ |y_i − ŷ_i|. RMSE = sqrt((1/n) Σ (y_i − ŷ_i)^2). Lower MAE/RMSE is better for prediction errors. For classification, higher F1-score is generally better, especially when classes are imbalanced.

TASK 5 – Conduct at Least Three Controlled Tuning Experiments

Experiment	Change one factor	Keep constant	Record
E1: Hidden size	Compare 32 and 64 units	Dataset, split, optimizer and epochs	Validation metric and training time
E2: Learning rate	Compare 0.001 and 0.0005	Architecture and other settings	Convergence and validation metric
E3: Dropout	Compare 0.0 and 0.3	Architecture, split and optimizer	Training/validation gap and validation metric
Optional E4: Sequence length	Compare two reasonable lengths	Model and split	Metric and runtime

Change one factor at a time where practical. This makes it easier to explain which setting caused the observed change.

TASK 6 – Record Experimental Results

Model / experiment	Validation metric	Test metric	Training time	Notes
Simple RNN	Enter measured value	Enter measured value	Enter measured value	

LSTM or GRU	Enter measured value	Enter measured value	Enter measured value	
Deep recurrent model	Enter measured value	Enter measured value	Enter measured value	
Tuning experiment 1	Enter measured value	If selected, enter value	Enter measured value	
Tuning experiment 2	Enter measured value	If selected, enter value	Enter measured value	
Tuning experiment 3	Enter measured value	If selected, enter value	Enter measured value	

TASK 7 – Compare and Interpret the Models

Discuss whether the gated model performs better than the simple RNN, and whether adding recurrent layers provides a useful improvement. A more complex model may take longer to train and may overfit when the dataset is small. Compare the learning curves and actual measured metrics rather than assuming one architecture always wins.

TASK 8 – Explain When a Recurrent Network Is Preferable

Recurrent networks are useful when the order of the input matters, such as text, speech, sensor readings and time-series data. A gated RNN can be preferable when useful information appears far back in the sequence. A simple RNN may be sufficient for short and simple sequences. A deep recurrent model is worth using when extra capacity improves validation performance enough to justify its extra computation.

TASK 9 – Conclusion and References

The experiment compares model capacity and tuning choices on a consistent sequence task. The final model should be selected using validation performance and then evaluated on the test set. Cite the dataset and official framework documentation.